Comparative Analysis of AdamW, Muon, and Hybrid Optimizers for Parameter-Efficient Fine-Tuning of Large Language Models

Rshtuni V.

March 23, 2026

Abstract

This report presents a detailed study of three gradient descent optimization strategies for parameter-efficient fine-tuning (LoRA) of large language models. We examine: the classical adaptive optimizer AdamW, the modern Muon optimizer based on matrix orthogonalization (implementation based on `MoonshotAI/Moonlight` [3]), and a proposed hybrid approach combining the advantages of both methods. Implementation of the MeZO optimizer was optional (challenge) and was not completed in this study. For each optimizer, we provide mathematical formulations, pseudocode algorithms, and computational complexity analysis. Experiments were conducted on the Qwen2.5-0.5B model using the OpenWebText dataset (1%) and evaluation on the PIQA logical reasoning task. Results show that AdamW provides the best training speed (0.682 steps/s), Muon saves ∼2% GPU memory, and the hybrid approach demonstrates balanced characteristics. All methods achieve comparable quality on the test set (∼70% accuracy).

Contents

1 Introduction **3**
1.1 Problem Relevance . . . 3
1.2 Problem Statement . . . 3
1.3 Report Structure . . . 4

2 Theoretical Foundations **5**
2.1 Training Problem Formulation . . . 5
2.2 LoRA: Low-Rank Adaptation . . . 5
2.3 Limitations of Standard Stochastic Gradient Descent . . . 5

3 Optimizer Descriptions **6**
3.1 AdamW: Adaptive Moment Estimation with Weight Decay Correction . . . 6
3.1.1 Method Idea . . . 6
3.1.2 Mathematical Formulation . . . 6
3.1.3 Pseudocode . . . 7
3.1.4 Computational Complexity and Memory . . . 7
3.2 Muon: Optimization via Matrix Orthogonalization . . . 7
3.2.1 Method Idea . . . 7
3.2.2 Mathematical Formulation . . . 7
3.2.3 Pseudocode . . . 9
3.2.4 Computational Complexity and Memory . . . 9
3.3 Hybrid: Combined Approach . . . 9
3.3.1 Motivation . . . 9
3.3.2 Parameter Partitioning Strategy . . . 10
3.3.3 Hybrid Optimizer Pseudocode . . . 10
3.3.4 Advantages and Limitations . . . 10

4 Experimental Setup **12**
4.1 Hardware and Software . . . 12
4.2 Model and Data . . . 13
4.3 Metrics and Evaluation Protocol . . . 13

5 Results **14**
5.1 Summary Metrics Table . . . 14
5.2 Performance Analysis . . . 14

6 Conclusion **16**

1 Introduction

1.1 Problem Relevance

Modern large language models (LLMs) contain hundreds of millions to billions of parameters, making their full fine-tuning computationally expensive. Parameter-efficient fine-tuning methods, such as LoRA (Low-Rank Adaptation) [1], enable adaptation of pre-trained models to new tasks by updating only a small fraction of parameters. According to [1], when applied to GPT-3 175B, the number of trainable parameters can be as low as **0.01%** of the total, while in experiments with RoBERTa and DeBERTa it ranges from **0.24%** to **0.31%**.

A key component of the training process is the choice of optimizer — an algorithm that determines the direction and magnitude of model weight updates based on loss function gradients. In recent years, new optimization methods have emerged, claiming to improve convergence and resource efficiency compared to classical approaches.

1.2 Problem Statement

This study conducts a comparative analysis of optimizers for LoRA fine-tuning according to the technical specification:

1. **AdamW** [2] — standard adaptive method with weight decay correction (base task);
2. **Muon** [3] — optimizer based on matrix orthogonalization (base task);
3. **Hybrid** — author's hybrid approach combining Muon and AdamW (challenge task);
4. **MeZO** [4] — gradient-free optimizer (challenge task, not implemented).

Objective: Conduct a comparative analysis of the three implemented optimizers according to the following criteria:

- Training time;
- Peak GPU memory consumption;
- Final loss function value;
- Accuracy on the test task (PIQA);
- Convergence speed.

Note on MeZO: Implementation of the MeZO optimizer was optional (challenge) within the technical specification and was not completed in this study. Priority was given to detailed analysis of AdamW, Muon, and their hybrid combination, enabling a more in-depth comparison of the primary methods.

1.3 Report Structure

The report is organized as follows:

- Section 2 — theoretical foundations of training and LoRA;
- Section 3 — optimizer descriptions with formulas and pseudocode;
- Section 4 — experimental setup;
- Section 5 — results and analysis;
- Section 6 — conclusions and recommendations;
- Appendix — instructions for reproducing results.

2 Theoretical Foundations

2.1 Training Problem Formulation

Let a model with parameters $\theta \in \mathbb{R}^d$ and a loss function $\mathcal{L}(\theta; \mathcal{D})$ computed on training dataset $\mathcal{D}$ be given. The training objective is to find parameters θ^* that minimize empirical risk:

$$\theta^* = \arg\min_{\theta} \mathbb{E}_{(x,y)\sim\mathcal{D}}[\ell(f_\theta(x), y)], \tag{1}$$

where ℓ is the loss function for a single example, and f_θ is the model prediction.

2.2 LoRA: Low-Rank Adaptation

In the context of LoRA fine-tuning, the original weights W_0 are frozen, and a low-rank increment is added:

$$W = W_0 + \Delta W = W_0 + BA, \quad A \in \mathbb{R}^{r\times k},\ B \in \mathbb{R}^{n\times r},\ r \ll \min(n, k). \tag{2}$$

Only matrices A and B are trained, substantially reducing the number of trainable parameters.

LoRA advantages:

- Significant reduction in trainable parameters (down to 0.01–0.3% of total [1]);
- No inference latency overhead (adapters can be merged with base weights);
- Ability to store multiple adapters for different tasks.

2.3 Limitations of Standard Stochastic Gradient Descent

Classical SGD updates parameters according to:

$$\theta_{t+1} = \theta_t - \eta \cdot g_t, \quad g_t = \nabla_\theta \mathcal{L}(\theta_t), \tag{3}$$

where η is the learning rate.

Main limitations of SGD:

- **Sensitivity to gradient scale**: parameters with different scales require different learning steps;
- **Mini-batch noise**: gradient stochasticity may cause trajectory oscillations;
- **Lack of adaptation**: a single learning rate for all parameters is inefficient for sparse or rarely updated weights.

These limitations motivate the use of adaptive optimization methods.

3 Optimizer Descriptions

3.1 AdamW: Adaptive Moment Estimation with Weight Decay Correction

3.1.1 Method Idea

AdamW [2] combines advantages of two approaches:

- **Momentum**: accumulation of exponentially decaying gradient averages to smooth updates;
- **Adaptive scaling**: normalization of gradients by their variance estimates for automatic per-parameter step adaptation;
- **Weight decay decoupling**: application of regularization separately from the gradient step, improving generalization.

3.1.2 Mathematical Formulation

At step t, for each parameter θ_i, compute:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \qquad \text{(first moment)} \tag{4}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \qquad \text{(second moment)} \tag{5}$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t} \qquad \text{(bias correction)} \tag{6}$$

$$\hat{v}_t = \frac{v_t}{1-\beta_2^t} \qquad \text{(bias correction)} \tag{7}$$

$$\theta_{t+1} = \theta_t - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_t \right) \qquad \text{(update with weight decay)} \tag{8}$$

where:

- g_t — gradient at step t;
- $\beta_1 = 0.9$, $\beta_2 = 0.999$ — momentum decay coefficients (standard values for pure AdamW per [2]);
- $\epsilon = 10^{-8}$ — small constant for numerical stability;
- λ — weight decay coefficient;
- η — base learning rate.

3.1.3 Pseudocode

Algorithm 1 AdamW

Require: Parameters θ, gradients g, hyperparameters $\eta, \beta_1, \beta_2, \epsilon, \lambda$

Initialize $m \leftarrow 0$, $v \leftarrow 0$, $t \leftarrow 0$
while stopping criterion not met **do**
  $t \leftarrow t + 1$
  $g \leftarrow \nabla_\theta \mathcal{L}(\theta)$ ▷ Compute gradient
  $m \leftarrow \beta_1 \cdot m + (1 - \beta_1) \cdot g$
  $v \leftarrow \beta_2 \cdot v + (1 - \beta_2) \cdot g^2$
  $\hat{m} \leftarrow m/(1 - \beta_1^t)$
  $\hat{v} \leftarrow v/(1 - \beta_2^t)$
  $\theta \leftarrow \theta - \eta \cdot \left(\frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon} + \lambda \cdot \theta \right)$
end while
return θ

3.1.4 Computational Complexity and Memory

- **Memory**: $2d$ additional scalars per parameter (m and v);
- **Computations**: $O(d)$ operations per step (element-wise operations);
- **Parallelization**: trivial, all operations are element-wise.

3.2 Muon: Optimization via Matrix Orthogonalization

3.2.1 Method Idea

Muon (Matrix Orthogonalization for Updating Networks) [3] is based on the hypothesis that the gradient direction can be improved by orthogonalizing the update matrix. The method applies Newton-Schulz iterations to approximate the polar decomposition of the gradient matrix, theoretically accelerating convergence in tasks with correlated gradients.

Key implementation features:

- Applied only to **2D tensors** (weight matrices), excluding embeddings and `lm_head`;
- Uses **a single moment buffer** instead of two, saving memory;
- Orthogonalization performed via **quintic Newton-Schulz iterations** with coefficients $(a, b, c) = (3.4445, -4.7750, 2.0315)$;
- Default number of iterations: $K = 5$ (empirically chosen as balance between accuracy and speed);
- Automatic learning rate adjustment based on matrix size: $\eta_{\text{adj}} = \eta \cdot 0.2 \cdot \sqrt{\max(n, m)}$.

3.2.2 Mathematical Formulation

Let $G \in \mathbb{R}^{n \times m}$ be the gradient matrix for a weight layer.

Note on hyperparameters:

- For **pure AdamW**, standard values $\beta_1 = 0.9$, $\beta_2 = 0.999$ are used;
- In the **Muon** implementation (based on `MoonshotAI/Moonlight`), internal updates for 1D parameters and fallback logic use $\beta_1 = 0.9$, $\beta_2 = 0.95$;
- In **hybrid mode**, parameters processed via the AdamW component inherit the same $\beta_1 = 0.9$, $\beta_2 = 0.95$ for consistency with the code.

Step 1: Normalization and preparation

$$X_0 = \frac{G}{\|G\|_F + \epsilon}, \quad \text{if } n > m, \text{ then } X_0 \leftarrow X_0^\top \tag{9}$$

Step 2: Quintic Newton-Schulz iterations
For $k = 0, 1, \ldots, K-1$:

$$A_k = X_k X_k^\top \tag{10}$$
$$B_k = b \cdot A_k + c \cdot A_k^2 \tag{11}$$
$$X_{k+1} = a \cdot X_k + B_k X_k \tag{12}$$

where coefficients $(a, b, c) = (3.4445, -4.7750, 2.0315)$ are tuned to maximize convergence speed [3].

Step 3: Update with momentum and step adjustment

$$M_t = \beta M_{t-1} + (1-\beta) \cdot X_K \tag{13}$$
$$\eta_{\text{adj}} = \eta \cdot 0.2 \cdot \sqrt{\max(n, m)} \tag{14}$$
$$\theta_{t+1} = \theta_t - \eta_{\text{adj}} \cdot M_t - \eta \cdot \lambda \cdot \theta_t \tag{15}$$

3.2.3 Pseudocode

Algorithm 2 Muon: Update of a single weight matrix parameter

Require: Weight matrix W, gradient G, hyperparameters $\eta, \beta, \lambda, K, (a, b, c)$

```
if G is not a 2D tensor then
    return AdamW update for G                          ▷ Fallback for vectors
end if
                                                       ▷ Step 1: normalization
X ← G/(‖G‖_F + ε)
if rows(G) > cols(G) then
    X ← X^⊤
end if
                                                       ▷ Step 2: quintic Newton-Schulz iterations
for k = 1 to K do
    A ← X · X^⊤
    B ← b · A + c · (A · A)
    X ← a · X + B · X
end for
if rows(G) > cols(G) then
    X ← X^⊤
end if
                                                       ▷ Step 3: momentum and update
M ← β · M + (1 − β) · X
η_adj ← η · 0.2 · √(max(shape(W)))
W ← W − η_adj · M − η · λ · W
return W
```

3.2.4 Computational Complexity and Memory

- **Memory**: d scalars per parameter (one moment M) + temporary buffers $O(nm)$ for iterations;
- **Computations**: $O(K \cdot nm \cdot \min(n, m))$ per step due to matrix multiplications in Newton-Schulz iterations;
- **Parallelization**: matrix operations parallelize well on GPU, but iterative nature limits pipelining.

3.3 Hybrid: Combined Approach

3.3.1 Motivation

Neither AdamW nor Muon is universally optimal:

- AdamW is fast and stable but may converge slower in tasks with strongly correlated gradients;
- Muon theoretically accelerates convergence but requires more computation and may be unstable in early training stages.

The hybrid approach idea: leverage strengths of each method by assigning them to different parameter groups.

3.3.2 Parameter Partitioning Strategy

Model parameters are split into two groups according to the algorithm in `src/optimizers/optimizer.p`

1. **2D parameter filtering**: from all parameters, select only 2D tensors, excluding those containing `embed_tokens` or `lm_head` in their name:

```
all_2d_params = [
    p for p in params
    if p.ndim >= 2
    and "embed_tokens" not in str(p)
    and "lm_head" not in str(p)
]

```

2. **Half-split**: first 50% of selected 2D parameters are trained via Muon, remaining 50% + all 1D parameters — via AdamW:

```
split_idx = len(all_2d_params) // 2
muon_params = all_2d_params[:split_idx]
adamw_params = all_2d_params[split_idx:] + other_params

```

3.3.3 Hybrid Optimizer Pseudocode

Algorithm 3 Hybrid: Combined update of all parameters

Require: Parameters $\Theta = \{\theta_1, \ldots, \theta_N\}$, gradients $\{g_1, \ldots, g_N\}$

$\triangleright$ Filter 2D parameters

$all_2d \leftarrow \{p \in \Theta \mid p.\text{ndim} \geq 2 \wedge \text{"embed_tokens"} \notin p \wedge \text{"lm_head"} \notin p\}$

$other \leftarrow \Theta \setminus all_2d$

$\triangleright$ Split in half

$split \leftarrow \lfloor |all_2d|/2 \rfloor$

$muon_params \leftarrow all_2d[0 : split]$

$adamw_params \leftarrow all_2d[split :] \cup other$

$\triangleright$ Update via unified Muon interface

return MUON.STEP($muon_params$, $adamw_params$, g)

3.3.4 Advantages and Limitations

Advantages:

- **Speed-quality balance**: critically important layers receive more powerful optimization (Muon), others — fast (AdamW);
- **Flexibility**: partitioning strategy can be adapted to architecture;
- **Backward compatibility**: when $split = 0$, the method degenerates to pure AdamW.

Limitations:

- **Heuristic partitioning**: selecting first 50% of matrices may be suboptimal for all architectures;

- **Added complexity**: requires tracking parameter types and ordering;
- **No theoretical guarantees**: combining two optimizers lacks formal convergence proof.

4 Experimental Setup

4.1 Hardware and Software

Table 1: Hardware Configuration

Component	**Specifications**
CPU	Intel Core i5-11400F (6 cores / 12 threads)
RAM	16 GB DDR4
GPU	NVIDIA GeForce RTX 5060 Ti, 16 GB VRAM*
Operating System	Windows 10 Pro

**Note*: The specified GPU is beyond publicly available specifications at the time of writing; characteristics are provided as reported by the experiment author. bfloat16 support is assumed based on architecture.

Table 2: Software Dependencies (from `requirements.txt`)

Package	**Version**
torch	2.10.0+cu128
datasets	4.8.2
hydra-core	1.3.2
lm_eval	0.4.11
matplotlib	3.10.8
mypy	1.19.1
peft	0.18.1
psutil	7.2.2
pylint	4.0.5
tokenizers	0.22.2
transformers	5.3.0
trl	0.29.0

4.2 Model and Data

Table 3: Experimental Setup Parameters

Component	**Value**
Model	Qwen/Qwen2.5-0.5B (0.49B parameters)
Adaptation	LoRA: $r = 16$, $\alpha = 32$, dropout=0.05
Target modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Trainable parameters	8.8M (1.75% of total)
Training dataset	Elriggs/openwebtext-100k (1% = 1000 samples)
Test dataset	PIQA (evaluation on test subset; original: 3000 examples)
Computational precision	bfloat16 (assumed support on target platform)
Batch size	4 per device, 4 gradient accumulation steps
Epochs	5
Learning rate	5×10^{-5}, cosine schedule with 3% warmup
Weight decay	0.01 (AdamW, Muon), 0.1 (Hybrid)
Optimizers	AdamW / Muon / Hybrid

4.3 Metrics and Evaluation Protocol

For each experiment, we record:

1. **Training time**: from start of first step to completion of final epoch;
2. **Peak GPU memory**: maximum value observed;
3. **Final loss**: average loss value at final step;
4. **PIQA Accuracy**: fraction of correct answers on test dataset after fine-tuning;
5. **Steps per second**: averaged mini-batch processing speed.

All experiments are run with fixed random seed (42) for reproducibility.

5 Results

5.1 Summary Metrics Table

Table 4: Optimizer Comparison by Key Metrics

Metric	AdamW	Muon	Hybrid
Training time	**7m 41s**	9m 22s	8m 23s
Peak memory, MB	3912	**3846**	3878
Final loss	**2.863**	2.903	2.888
PIQA Accuracy, %	**70.29**	70.13	69.86
Steps per second	**0.682**	0.560	0.626
Trainable params	8.8M (1.75% of total)		

Table 5: Optimizer Hyperparameters Used in Experiments

Parameter	AdamW	Muon	Hybrid
β_1	0.9	0.9	0.9
β_2	0.999	0.95	0.95
ϵ	10^{-8}	10^{-7}	10^{-7}
Weight decay (λ)	0.01	0.01	0.1
Newton-Schulz steps (K)	—	5	5
LR adjustment factor	—	$0.2 \cdot \sqrt{\max(n, m)}$	$0.2 \cdot \sqrt{\max(n, m)}$

Note on reproducibility: All experiments were run with fixed seed=42. Small metric variations (±0.1% accuracy) may occur across repeated runs due to non-deterministic operations in cuBLAS and CUDA asynchrony.

5.2 Performance Analysis

Training speed. AdamW demonstrates best performance: 0.682 steps/s vs. 0.560 for Muon (−18%). This is explained by absence of Newton-Schulz iterations requiring additional matrix multiplications. The hybrid approach occupies an intermediate position (0.626 steps/s), as only half of parameters are processed by the heavier Muon.

Memory consumption. Muon saves ∼66 MB (∼1.7%) GPU memory compared to AdamW by using a single moment buffer instead of two. The hybrid method consumes intermediate memory, proportional to the fraction of parameters processed by each optimizer.

Training quality. All three methods achieve comparable final loss (∼2.86–2.90) and PIQA accuracy (∼70%). Small differences (less than 0.5% accuracy) are statistically insignificant given the data volume (1000 training samples). This indicates that for small-scale tasks, optimizer choice primarily affects efficiency rather than final quality.

Convergence dynamics. Log analysis shows all optimizers exhibit monotonic loss decrease without divergence signs. In early stages (first 2 epochs), Muon sometimes shows sharper oscillations, possibly due to iterative nature of orthogonalization. By training end, trajectories of all methods stabilize.

6 Conclusion

This work conducted a detailed comparative analysis of three optimizers for LoRA fine-tuning of large language models:

1. **AdamW** remains a reliable choice for most practical tasks: it is fast, stable, and well-studied. Its per-parameter adaptivity effectively handles multi-scale gradients, while weight decay correction improves generalization.

2. **Muon** offers an innovative approach via matrix orthogonalization, theoretically accelerating convergence in tasks with correlated gradients. The implementation is adapted from the `MoonshotAI/Moonlight` repository [3]. In practice, it saves memory and shows comparable quality but requires more computation.

3. **Hybrid** demonstrates that method combination can yield balanced results. The proposed parameter partitioning strategy (first 50% of 2D matrices → Muon, rest → AdamW) is a simple yet effective heuristic.

Key finding: for small-to-medium scale tasks (as in this experiment), differences in final quality between optimizers are minimal. Selection should be based on time and memory constraints: AdamW for speed, Muon for resource savings, Hybrid for balance.

Work limitations: Implementation of the MeZO optimizer [4] was optional (challenge) within the technical specification and was not completed in this study. Priority was given to detailed analysis of AdamW, Muon, and their hybrid combination.

References

References

[1] Hu E. J. et al. *LoRA: Low-Rank Adaptation of Large Language Models.* arXiv preprint arXiv:2106.09685, 2021.

[2] Loshchilov I., Hutter F. *Decoupled Weight Decay Regularization.* International Conference on Learning Representations (ICLR), 2019.

[3] Liu J. et al. *Muon is Scalable for LLM Training.* arXiv preprint arXiv:2502.16982, 2025. `https://github.com/MoonshotAI/Moonlight`

[4] Malladi S. et al. *MeZO: Memory-Efficient Zeroth-Order Optimizer for LLM Fine-tuning.* arXiv preprint arXiv:2305.17333, 2023.

[5] Bisk Y. et al. *PIQA: Reasoning about Physical Commonsense in Natural Language.* Proceedings of the AAAI Conference on Artificial Intelligence, 2020.

[6] Schulz G. *Iterative Berechnung der reziproken Matrix.* Zeitschrift für Angewandte Mathematik und Mechanik, 1933.